

```
create table dept  
(  
dno integer primary key,  
dname varchar(30),  
total_emp integer,  
total_sal integer);
```

```
insert into dept values(101,'research',0,0);  
insert into dept values(102,'account',0,0);
```

```
CREATE TABLE emp_table ( empid int, empname name, salary  
int ,dnum int, primary key(empid), foreign key(dnum)  
references dept(dno));
```

```
CREATE FUNCTION inse_function() RETURNS trigger AS '  
BEGIN
```

```
IF tg_op = "INSERT" THEN  
    UPDATE dept set total_emp=total_emp + 1,  
        total_sal=total_sal+ new.salary where dept.dno =  
new.dnum;  
    RETURN new;  
END IF;
```

```
END  
' LANGUAGE plpgsql;
```

```
CREATE TRIGGER emp_update AFTER INSERT  
ON emp_table FOR each ROW  
EXECUTE PROCEDURE inse_function();
```

```
select *from dept;
```

```
"dno"|"dname"|"total_emp"|"total_sal"  
101|"research"|0|0  
102|"account"|0|0
```

```
insert into emp_table values(1,'amit',2000,101);  
insert into emp_table values(2,'sachit',1000,102);
```

```
insert into emp_table values(3,'nikhil',5000,101);  
insert into emp_table values(4,'sunil',10000,102);
```

The table department will be updated with 2 employees in each department and total sal of corresponding employees

To check
select *from dept;

```
"dno"|"dname"|"total_emp"|"total_sal"  
101|"research"|2|7000  
  
102|"account"|2|11000
```

an example to demonstrate PostgreSQL create trigger AFTER INSERT statement.

See this example:

In the following example, we keep audit trial for every record being inserted in COMPANY table.

Create a table named COMPANY, by using the following query:

```
1. CREATE TABLE COMPANY(  
2.   ID INT PRIMARY KEY   NOT NULL,  
3.   NAME      TEXT   NOT NULL,  
4.   AGE       INT    NOT NULL,  
5.   ADDRESS   CHAR(50),  
6.   SALARY    REAL  
7.);
```

To keep audit trial, we will create a new table called AUDIT where log messages will be inserted whenever there is an entry in COMPANY table for a new record.

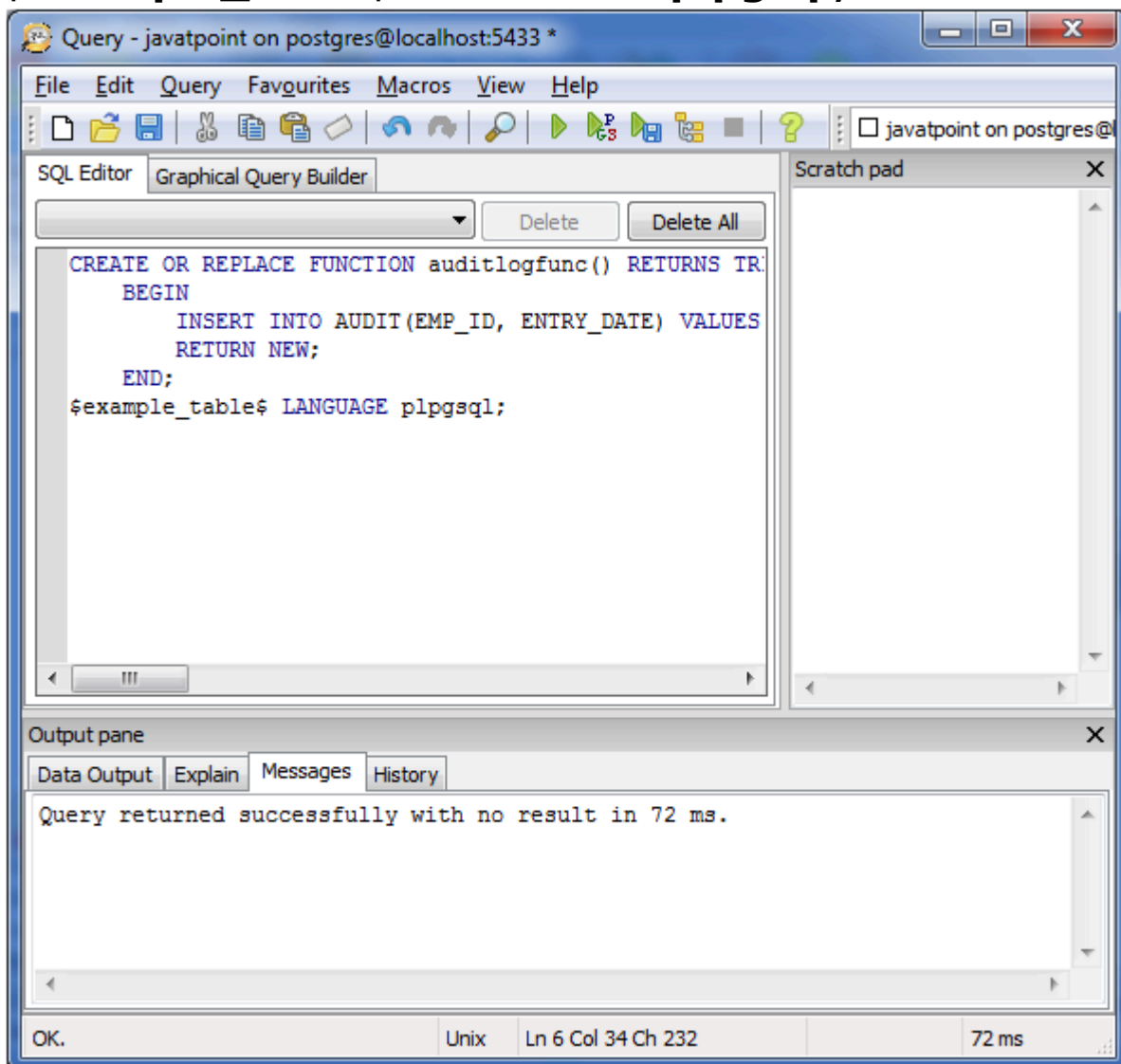
Create another table Audit, by using the following query:

```
1. CREATE TABLE AUDIT(  
2.   EMP_ID INT NOT NULL,  
3.   ENTRY_DATE TEXT NOT NULL  
4.);
```

Before creating a trigger on company table, first create a function/ procedure named auditlogfunc().

Execute the following query:

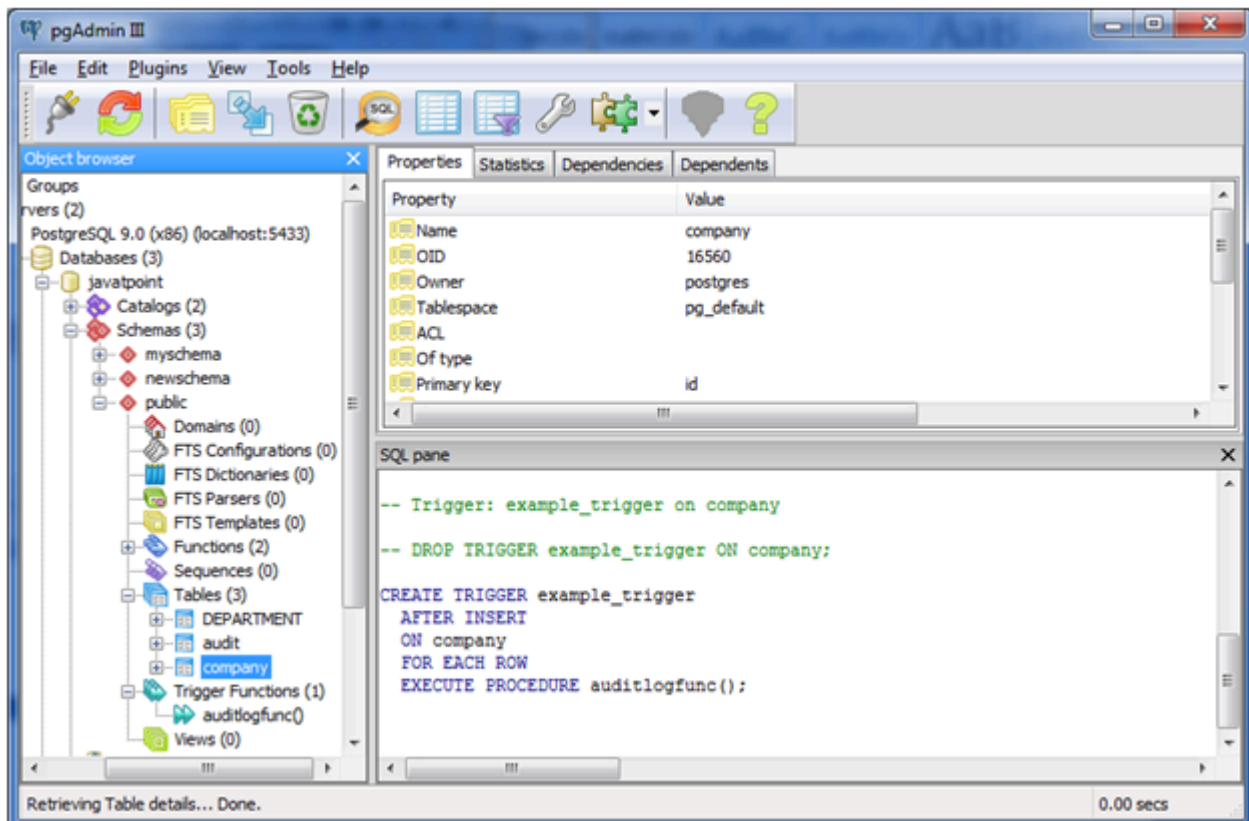
1. **CREATE OR REPLACE FUNCTION** auditlogfunc() **RETURNS TRIGGER AS** \$example_table\$
2. **BEGIN**
3. **INSERT INTO** AUDIT(EMP_ID, ENTRY_DATE) **VALUES** (new.ID, **current_timestamp**);
4. **RETURN NEW;**
5. **END;**
6. **\$example_table\$ LANGUAGE plpgsql;**



Now create a trigger on COMPANY table by using the following query:

Execute the following query:

1. **CREATE TRIGGER** example_trigger **AFTER INSERT ON** **COMPANY**
2. **FOR EACH ROW EXECUTE PROCEDURE** auditlogfunc()
;



Insert some values in the COMPANY table.

Edit Data - PostgreSQL 9.0 (x86) (localhost:5433) - javatpoint - company

File Edit View Tools Help

No limit

	id [PK] integer	name text	age integer	address character(50)	salary real
1	1	Rakul	32	Shimla	20000
2	2	Aryan	26	Mumbai	30000

Scratch pad

2 rows.

At the same time, two records will be created in AUDIT table. These records are the result of trigger, which we have created on AFTER INSERT on COMPANY table.

Edit Data - PostgreSQL 9.0 (x86) (localhost:5433) - javatpoint - audit

File Edit View Tools Help

No limit

	emp_id integer	entry_date text
1	1	2016-06-03
2	2	2016-06-03

Scratch pad

2 rows.